Módulos y paquetes

En programas reales, el código no se escribe en un único archivo. Python proporciona un sistema de módulos y paquetes que permite:

- Organizar el código en unidades reutilizables
- Evitar colisiones de nombres
- Facilitar el mantenimiento y la escalabilidad
- Distribuir y reutilizar software

1 MÓDULOS

Un módulo es cualquier archivo con extensión `.py`. Por ejemplo, si tubiéramos un fichero llamado `matematicas.py` con:

```
def suma(a, b):
    return a + b
```

Este archivo sería un módulo y podríamos importarlo desde otros programas.

1.1 IMPORTACIÓN DE MÓDULOS

Mediante la sentencia `import` se importa el módulo completo, permitiendo utilizar el conjunto de variables, métodos y clases que hubiera declarados en dicho módulos.

```
import matematicas

resultado = matematicas.suma(2, 3)
```

Otra alternativa es importar los nombres indicado de un determinado módulo, aunque esto puede provocar que los nombres importados coincidan con los que estamos utilizando en nuestro programa:

```
from matematicas import suma

resultado = suma(2, 3)
```

Para evitar colisiones de nombres o utilizar nombres que nos resulten más intuitivos podemos utilizar un alias con `as`.

```
import matematicas as m
```

2 PUNTO DE ENTRADA DEL PROGRAMA

Ahora que existe se comienza a trabajar con módulos es importante diferenciar el comportamiento de un fichero Python dependiendo de si lo estamos trabajando como módulo o como programa principal. Para ello se utiliza la siguiente sentencia:

```
if __name__ == "__main__":
```

!! `__name__` es una variable especial que Python define automáticamente y dependiendo si el archivo se ejecuta directamente, en cuyo caso se le asignará como valor `__main__` o el nombre del archivo, en caso de que se importe como módulo.

Esta construcción permite distinguir si un archivo Python se está ejecutando directamente o si está siendo importado como módulo.

```
def main():
    print("Ejecutando el programa principal")

if __name__ == "__main__":
    main()
```

3 PAQUETES

Un paquete es un conjunto de módulos organizados en directorios. Cuando dentro de un directorio existe un fichero `__init__.py` indica que el directorio es un paquete. El archivo `__init__.py` se ejecuta al importar el paquete y puede usarse para inicializar el paquete, exponer una API[1] pública u ocultar detalles internos.
Por ejemplo, prodiamos tener un directorio denominado `paquetetito` con un fichero `__init__.py` y un fichero que sea `matematicas.py`.

```
# paquetetio/__init__.py
from .matematicas import suma

__all__ = ["suma"]
```

[1]Una API (*Application Programming Interface*) es una carta de presentación de lo que un paquete puede hacer y una especificación de como puede hacerlo. Si queremos comunicarnos con ese módulo, será utilizando dicha API y según los métodos que esta tenga especificada.

Esto permitiría que desde otro fichero que alcance este paquete pueda importarlo tal que:

```
from paquetito import suma
```

El atributo especial `__all__` permite controlar la interfaz pública de un paquete, especificando explícitamente los nombres que se importarán al utilizar `from paquetito import *`. En ausencia de `__all__`, Python exporta todos los identificadores definidos o importados en el archivo `__init__.py` cuyo nombre no comienza por `_`.

4 EL SISTEMA DE IMPORTACIÓN DE PYTHON

Cuando Python ejecuta un `import`, busca el módulo en este orden:

1. **Directorio del script principal**. Python comienza buscando el módulo en el directorio donde se encuentra el script que se está ejecutando. Si un archivo tiene el mismo nombre que un módulo estándar (por ejemplo, `utils.py`), Python importará el archivo local en lugar del módulo estándar, lo que puede provocar errores difíciles de detectar.
2. **Directorios listados en `sys.path`**. Si el módulo no se encuentra en el directorio del script principal, Python continúa la búsqueda en los directorios incluidos en la lista `sys.path`. Esta lista se inicializa automáticamente al arrancar Python e incluye:
   - El directorio del script principal
   - Directorios definidos en la variable de entorno `PYTHONPATH`
   - Directorios específicos de la instalación de Python
3. **Librerías estándar**: A continuación, Python busca el módulo en la biblioteca estándar, que incluye módulos distribuidos junto con Python.
4. **`site-packages`** Por último, Python busca el módulo en el directorio `site-packages`, donde se instalan las librerías externas mediante herramientas como `pip`. Cada entorno virtual tiene su propio directorio `site-packages`, lo que permite aislar dependencias entre proyectos.
   - Por ejemplo, si quisiéramos instalar una librería para realizar peticiones web, podríamos utilizar `requests`, la cual no viene integrada con Python y hay que instalar con la siguiente sentencia:

     ```
     pip install requests
     ```

!! Si una librería no está instalada en el entorno activo, Python lanzará un `ModuleNotFoundError`.

5 ENTORNOS VIRTUALES

Los entornos virtuales son entornos Python que aíslan dependencias, evitando conflictos entre proyectos y permitiendo una mayor reproductibilidad.
Para crear un entorno virtual es necesario utilizar `venv` que es el gestor oficial de Python de entornos virtuales[2]:

```
python -m venv entorno
```

Una vez creado tenemos que activarlo, la sentencia de activación es diferente si utilizamos Linux/macOS o Windows:

```
source entorno/bin/activate
```

```
venv\Scripts\activate
```

!!

Habitualmente para documentar las dependencias existe un fichero `requirements.txt` donde almacenamos el nombre de las librerías que se han instalado junto a la versión de cara a poder simular mejor el entorno que se está utilizando en otros equipos. Por ejemplo, si tuviéramos un fichero `requirements.txt` tal que:

```
requests==2.31.0
numpy>=1.25
```

Podemos realizar la instalación en otro entorno con solamente moviendo el fichero `requirements.txt` de un entorno a otro.

```
pip install -r requirements.txt
```

Existen otros gestores de módulos para Python con sus propios comandos y sus propias características:

- uv
- poetry
- pyenv
- conda
- pipenv
- hatch

[2]Puede ser que si utilizas sistemas UNIX no lo tengas instalado, en dicho caso, instala el venv propio de la verisón de Python que estés utilizando.